

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“Relasi Antar Objek (Class Diagram & Asosiasi)”



disusun Oleh:
NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:
NURFIAH, S.ST., M.Kom.

Asisten Praktikum:

Siti Fadhilah Rahmi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan salah satu konsep penting dalam pembuatan program karena program dapat dibangun menggunakan objek yang memiliki data dan perilaku. Salah satu konsep dalam PBO adalah hubungan atau relasi antar objek. Relasi tersebut memungkinkan sebuah objek berhubungan dengan objek lainnya untuk menjalankan suatu proses dalam program.

Pada praktikum ini, konsep relasi antar objek diterapkan pada studi kasus Mini Bank. Program sebelumnya dapat melakukan transaksi setor dan tarik tunai, tetapi belum menyimpan riwayat dari setiap transaksi yang dilakukan. Oleh karena itu, program dikembangkan dengan menambahkan class Transaksi yang digunakan untuk menyimpan data setiap transaksi. Setiap objek Rekening memiliki banyak objek Transaksi, sehingga terbentuk relasi one-to-many. Konsep ini sesuai dengan tujuan praktikum untuk memahami asosiasi dan pengelolaan kumpulan objek menggunakan ArrayList.

Pada pengembangan program ini, setiap transaksi setor akan dicatat sebagai transaksi Kredit, sedangkan transaksi penarikan dicatat sebagai transaksi Debit. Riwayat tersebut kemudian disimpan di dalam ArrayList<Transaksi> pada class Rekening. Selain itu, program juga dilengkapi fitur untuk menampilkan seluruh riwayat transaksi melalui method cetakMutasi().

Melalui praktikum ini, mahasiswa dapat memahami bagaimana suatu class dapat memiliki hubungan dengan class lainnya serta bagaimana objek dapat dibuat dan disimpan di dalam objek lain. Dengan penerapan tersebut, konsep asosiasi dan relasi one-to-many dapat dipahami melalui contoh program yang sederhana dan lebih nyata.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep relasi antar objek dalam Pemrograman Berorientasi Objek.
2. Memahami konsep asosiasi one-to-many antara Rekening dan Transaksi.
3. Menerapkan ArrayList untuk menyimpan banyak objek Transaksi.
4. Membuat dan menggunakan objek Transaksi pada class Rekening.
5. Mencatat riwayat transaksi setor dan tarik tunai.
6. Menerapkan fitur cetak mutasi rekening menggunakan perulangan for-each.
7. Memahami penerapan relasi antar class dalam program Java sederhana.

BAB II PEMBAHASAN

2.1 Rekening.java

Class Rekening digunakan untuk menyimpan data rekening sekaligus mengelola transaksi yang dilakukan oleh rekening tersebut.

```
package MiniBank.copy;

import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new Locale("id",
"ID"));

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        // Wajib menginstalikasi ArrayList didalam constructor agar tidak
        NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek transaksi di dalam method)
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal+ " berhasil. Saldo
saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {

        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
    }
}
```

```

        System.out.println("Saldo Akhir    : " + saldo);
    }

    // Method tarik tunai
    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println(
                "Transaksi Gagal: Minimal nominal penarikan Rp10000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            // Merekam riwayat penarikan
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Penarikan berhasil.");
            System.out.println("Jumlah penarikan: Rp" + nominal);
            System.out.println("Saldo sekarang: Rp" + saldo);
        }
    }

    public void cekMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            int mulai = Math.max(0, riwayatTransaksi.size() - 3);
            System.out.println("--- TRANSAKSI TERBARU ---");
            for (int i = mulai; i < riwayatTransaksi.size(); i++) {
                riwayatTransaksi.get(i).cetakDetail();
            }
        }
    }

    public void cetakTotalTransaksi() {

        double totalSetor = 0;
        double totalTarik = 0;

        for (Transaksi trx : riwayatTransaksi) {

            if (trx.jenis.equals("Kredit")) {
                totalSetor += trx.nominal;
            } else if (trx.jenis.equals("Debit")) {
                totalTarik += trx.nominal;
            }
        }

        NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

        System.out.println("--- TOTAL TRANSAKSI ---");
        System.out.println("Total Setor : " + rupiah.format(totalSetor));
        System.out.println("Total Tarik : " + rupiah.format(totalTarik));

        System.out.println("\n--- 3 TRANSAKSI TERBARU ---");
    }

```

```

int mulai = Math.max(0, riwayatTransaksi.size() - 3);

for (int i = mulai; i < riwayatTransaksi.size(); i++) {
    riwayatTransaksi.get(i).cetakDetail();
}

}
}

```

- **Bagian Relasi Antar Objek**

ArrayList<Transaksi> riwayatTransaksi;

➔ Baris tersebut merupakan bagian penting dalam praktikum karena menunjukkan bahwa satu objek Rekening dapat memiliki banyak objek Transaksi. Hal ini merupakan penerapan asosiasi one-to-many.

- **Inisialisasi Riwayat**

- **Method setorTunai()**

public void setorTunai(double nominal)

➔ Method ini digunakan untuk melakukan setor tunai.

saldo += nominal;

➔ Menambahkan nominal setor ke saldo rekening.

String idTrx = "TRX-S-" + System.currentTimeMillis();

➔ Membuat ID transaksi setor dengan prefix TRX-S-.

Transaksi trxBaru =

new Transaksi(idTrx, "Kredit", nominal);

➔ Membuat objek transaksi baru dengan jenis Kredit.

riwayatTransaksi.add(trxBaru);

➔ Memasukkan transaksi tersebut ke dalam riwayat rekening.

- **Method tarikTunai()**

if (riwayatTransaksi.isEmpty())

➔ Digunakan untuk mengecek apakah rekening belum memiliki transaksi.

System.out.println(

"Belum ada transaksi pada rekening ini"

);

→ Menampilkan pesan apabila riwayat transaksi masih kosong.

Jika terdapat transaksi, program menggunakan:

for (Transaksi trx : riwayatTransaksi)

→ Perulangan for-each digunakan untuk mengambil setiap objek transaksi yang tersimpan di dalam riwayatTransaksi.

trx.cetakDetail();

→ Memanggil method cetakDetail() untuk menampilkan detail setiap transaksi.

2.2 Main.java

Class Main digunakan sebagai class utama untuk menjalankan program Mini Bank dan menyediakan menu bagi pengguna.

```
package MiniBank.copy;

import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;
        System.out.println("=== SISTEM PERBANKAN MINI ===");
        while (isRunning) {

            System.out.println("\nMenu Utama");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Total Transaksi");
            System.out.println("7. Keluar");
            System.out.print("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();

                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
```

```

        System.out.print("Masukkan Saldo Awal: ");
        double saldo = input.nextDouble();
        Rekening rekeningBaru =
            new Rekening(no, nama, saldo);
        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;
        System.out.println("Rekening berhasil ditambahkan.");
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening yang
tersedia.");
        } else {
            System.out.print(
                "Masukkan nomor rekening: "
            );
            String nomorCari = input.nextLine();
            boolean ditemukan = false;
            for (Rekening rekening : daftarRekening) {
                if (rekening.nomorRekening.equals(nomorCari)) {
                    akunAktif = rekening;
                    ditemukan = true;
                }
            }
        }
    }
}

```

```

        System.out.println("Akun berhasil diganti ke "
+ rekening.namaPemilik);
        break;
    }
    }
    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan.");
    }
    }
    break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            akunAktif.cetakTotalTransaksi();
        }
        break;

    case 7:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
}
input.close();
}
}

```

- **Import**

import java.util.Scanner;

import java.util.ArrayList;

➔ Scanner digunakan untuk menerima input pengguna.

➔ ArrayList digunakan untuk menyimpan beberapa objek Rekening.

- **Daftar Rekening**

ArrayList<Rekening> daftarRekening = new ArrayList<>();

➔ Digunakan untuk menyimpan beberapa rekening dalam program.

- **Akun Aktif**

Rekening akunAktif = null;

➔ Digunakan untuk menyimpan rekening yang sedang digunakan.

- **Menu Program**

- **Buka Rekening**

Rekening rekeningBaru =

new Rekening(no, nama, saldo);

→ Membuat objek rekening baru berdasarkan data yang dimasukkan pengguna.

daftarRekening.add(rekeningBaru);

→ Menyimpan rekening baru ke dalam daftarRekening.

akunAktif = rekeningBaru;

→ Menjadikan rekening yang baru dibuat sebagai akun aktif.

- **Setor Tunai**

akunAktif.setorTunai(setor);

→ Memanggil method setorTunai() pada rekening aktif.

- **Tarik Tunai**

akunAktif.tarikTunai(tarik);

→ Memanggil method tarikTunai() pada rekening aktif. Pada method inilah proses pencatatan transaksi Debit dari Tugas 1 dilakukan.

- **Cek Informasi**

akunAktif.cekInformasi();

→ Digunakan untuk menampilkan informasi rekening yang sedang aktif.

- **Ganti Akun**

for (Rekening rekening : daftarRekening)

→ Digunakan untuk mencari rekening berdasarkan data yang dimasukkan pengguna.

if (rekening.nomorRekening.equals(nomorCari))

→ Membandingkan nomor rekening yang dicari dengan nomor rekening yang tersedia.

akunAktif = rekening;

→ Jika ditemukan, rekening tersebut menjadi akun aktif.

- **Cetak Mutasi**

akunAktif.cetakMutasi();

→ Memanggil method cetakMutasi() untuk menampilkan seluruh riwayat transaksi rekening aktif.

- **Keluar**

isRunning = false;

→ Menghentikan perulangan menu sehingga program berakhir.

2.3 Transaksi.java

Class Transaksi digunakan untuk menyimpan informasi dari setiap transaksi yang dilakukan pada rekening. Dalam modul, class ini memiliki tiga atribut yaitu idTransaksi, jenis, dan nominal, serta method cetakDetail() untuk menampilkan detail transaksi.

```
package MiniBank.copy;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public void cetakDetail() {
        System.out.println("TD: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
}
```

- **String idTransaksi;**
Digunakan untuk menyimpan identitas atau kode transaksi.
- **String jenis;**
Digunakan untuk menyimpan jenis transaksi, yaitu Kredit untuk setor dan Debit untuk tarik.
- **double nominal;**
Digunakan untuk menyimpan jumlah uang yang ditransaksikan.
- **Constructor Transaksi(String idTransaksi, String jenis, double nominal)**
Digunakan untuk memberikan nilai awal pada atribut ketika objek transaksi dibuat.
- **this.idTransaksi = idTransaksi;**
Mengisi atribut idTransaksi dengan nilai yang diberikan.
- **cetakDetail()**

Digunakan untuk menampilkan informasi dari sebuah transaksi.

- **System.out.println(...)**

Menampilkan ID, jenis, dan nominal transaksi.

2.4 Tugas 1: Integrasi Rekam Jejak Penarikan

```
package MiniBank.copy;

import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new Locale("id", "ID"));

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        // Wajib menginstalasi ArrayList didalam constructor agar tidak
        // NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
        dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek transaksi di dalam method)
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal+ " berhasil. Saldo
            saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {

        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : " + saldo);
    }
}
```

```

    }

    // Method tarik tunai
    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println(
                "Transaksi Gagal: Minimal nominal penarikan Rp10000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            // Merekam riwayat penarikan
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Penarikan berhasil.");
            System.out.println("Jumlah penarikan: Rp" + nominal);
            System.out.println("Saldo sekarang: Rp" + saldo);
        }
    }

    public void cekMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            int mulai = Math.max(0, riwayatTransaksi.size() - 3);
            System.out.println("--- TRANSAKSI TERBARU ---");
            for (int i = mulai; i < riwayatTransaksi.size(); i++) {
                riwayatTransaksi.get(i).cetakDetail();
            }
        }
    }

    public void cetakTotalTransaksi() {

        double totalSetor = 0;
        double totalTarik = 0;

        for (Transaksi trx : riwayatTransaksi) {

            if (trx.jenis.equals("Kredit")) {
                totalSetor += trx.nominal;
            } else if (trx.jenis.equals("Debit")) {
                totalTarik += trx.nominal;
            }
        }

        NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

        System.out.println("--- TOTAL TRANSAKSI ---");
        System.out.println("Total Setor : " + rupiah.format(totalSetor));
        System.out.println("Total Tarik : " + rupiah.format(totalTarik));

        System.out.println("\n--- 3 TRANSAKSI TERBARU ---");

        int mulai = Math.max(0, riwayatTransaksi.size() - 3);

```

```

    for (int i = mulai; i < riwayatTransaksi.size(); i++) {
        riwayatTransaksi.get(i).cetakDetail();
    }
}
}

```

- **public void tarikTunai(double nominal) {**
→ Membuat method tarikTunai() yang digunakan untuk melakukan penarikan uang dari rekening. Parameter nominal berisi jumlah uang yang ingin ditarik.
- **if (nominal < 10000) {**
→ Mengecek apakah nominal penarikan kurang dari Rp10.000. Jika kurang, maka penarikan tidak diperbolehkan.
- **System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");**
→ Menampilkan pesan bahwa transaksi gagal karena nominal penarikan harus minimal Rp10.000.
- **} else if (nominal > saldo) {**
→ Mengecek apakah jumlah uang yang ingin ditarik lebih besar daripada saldo rekening. Jika lebih besar, maka saldo tidak mencukupi.
- **System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);**
→ Menampilkan pesan bahwa penarikan gagal karena saldo rekening tidak mencukupi.
- **} else {**
→ Bagian else dijalankan jika nominal penarikan sudah memenuhi syarat dan saldo mencukupi. Artinya, transaksi penarikan berhasil dilakukan.
- **saldo -= nominal;**
→ Mengurangi saldo rekening sesuai dengan jumlah uang yang ditarik.
- **String idTrx = "TRX-T-" + System.currentTimeMillis();**
→ Membuat ID untuk transaksi penarikan. ID diawali dengan "TRX-T-" sesuai dengan ketentuan tugas. System.currentTimeMillis() digunakan untuk menghasilkan angka berdasarkan waktu saat transaksi dilakukan sehingga ID transaksi dapat dibuat secara otomatis.
- **Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);**
→ Membuat objek Transaksi baru untuk menyimpan data transaksi penarikan.

- **riwayatTransaksi.add(trxBaru);**
→ Memasukkan objek transaksi yang baru dibuat ke dalam riwayatTransaksi. Dengan demikian, transaksi penarikan berhasil akan tersimpan dan dapat ditampilkan kembali sebagai riwayat transaksi.
- **System.out.println("Penarikan berhasil.");**
→ Menampilkan pesan bahwa penarikan berhasil dilakukan.
- **System.out.println("Jumlah penarikan: Rp" + nominal);**
→ Menampilkan jumlah uang yang berhasil ditarik.
- **System.out.println("Saldo sekarang: Rp" + saldo);**
→ Menampilkan saldo rekening setelah transaksi penarikan dilakukan.

2.5 Tugas 2: Fitur Cetak Mutasi Rekening

```
package MiniBank.copy;

import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new Locale("id", "ID"));

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;

        // Wajib menginstalasi ArrayList didalam constructor agar tidak
        // NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
        dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek transaksi di dalam method)
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal+ " berhasil. Saldo
            saat ini: Rp" + saldo);
        }
    }
}
```

```

    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

public void cekInformasi() {

    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : " + saldo);
}

// Method tarik tunai
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println(
            "Transaksi Gagal: Minimal nominal penarikan Rp10000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
    } else {
        saldo -= nominal;
        // Merekam riwayat penarikan
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Penarikan berhasil.");
        System.out.println("Jumlah penarikan: Rp" + nominal);
        System.out.println("Saldo sekarang: Rp" + saldo);
    }
}

public void cekMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        int mulai = Math.max(0, riwayatTransaksi.size() - 3);
        System.out.println("--- TRANSAKSI TERBARU ---");
        for (int i = mulai; i < riwayatTransaksi.size(); i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }
}

public void cetakTotalTransaksi() {

    double totalSetor = 0;
    double totalTarik = 0;

    for (Transaksi trx : riwayatTransaksi) {

        if (trx.jenis.equals("Kredit")) {
            totalSetor += trx.nominal;
        } else if (trx.jenis.equals("Debit")) {
            totalTarik += trx.nominal;
        }
    }
}

```

```

        NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

        System.out.println("--- TOTAL TRANSAKSI ---");
        System.out.println("Total Setor : " + rupiah.format(totalSetor));
        System.out.println("Total Tarik : " + rupiah.format(totalTarik));

        System.out.println("\n--- 3 TRANSAKSI TERBARU ---");

        int mulai = Math.max(0, riwayatTransaksi.size() - 3);

        for (int i = mulai; i < riwayatTransaksi.size(); i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }
}

```

- **public void cetakMutasi() {**
→ Membuat method cetakMutasi() yang digunakan untuk menampilkan riwayat transaksi rekening.
- **if (riwayatTransaksi.isEmpty()) {**
→ Mengecek apakah belum ada transaksi yang tersimpan dalam riwayatTransaksi.
- **System.out.println("Belum ada transaksi pada rekening ini");**
→ Menampilkan pesan jika rekening belum memiliki transaksi.
- **for (Transaksi trx : riwayatTransaksi) {**
→ Melakukan perulangan for-each untuk mengambil setiap objek transaksi yang terdapat dalam riwayatTransaksi.
- **trx.cetakDetail();**
→ Memanggil method cetakDetail() untuk menampilkan detail setiap transaksi.

Pada **Main.java**, ditambah menu:

- **System.out.println("6. Cetak Mutasi (Riwayat)");**
→ Menambahkan pilihan menu untuk melihat riwayat transaksi.
- **akunAktif.cetakMutasi();**
→ Memanggil method cekMutasi() dari rekening yang sedang aktif sehingga seluruh riwayat transaksi dapat ditampilkan.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep relasi antar objek dalam Pemrograman Berorientasi Objek dapat diterapkan melalui program Mini Bank. Relasi one-to-many diterapkan dengan membuat satu objek Rekening memiliki banyak objek Transaksi yang disimpan menggunakan ArrayList.

Pada Tugas 1, program dikembangkan agar setiap penarikan yang berhasil dapat dicatat sebagai transaksi dengan jenis Debit dan ID yang menggunakan prefix TRX-T-. Pada Tugas 2, program ditambahkan fitur cetakMutasi() untuk menampilkan seluruh riwayat transaksi menggunakan perulangan for-each.

Melalui praktikum ini, dapat dipahami bahwa relasi antar objek dapat membuat program lebih terstruktur karena data transaksi dapat dikelola sebagai bagian dari rekening. Penerapan ArrayList juga memudahkan program dalam menyimpan dan menampilkan banyak objek transaksi.